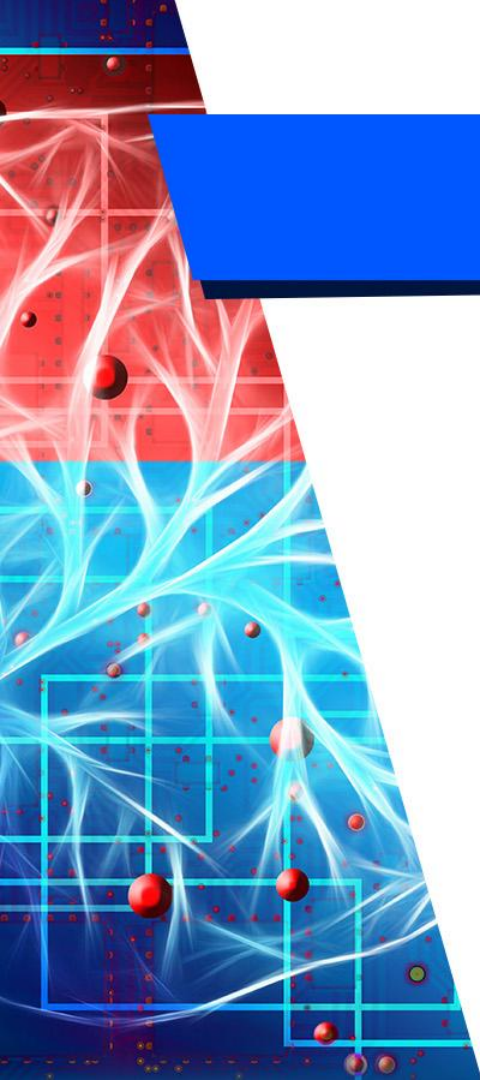
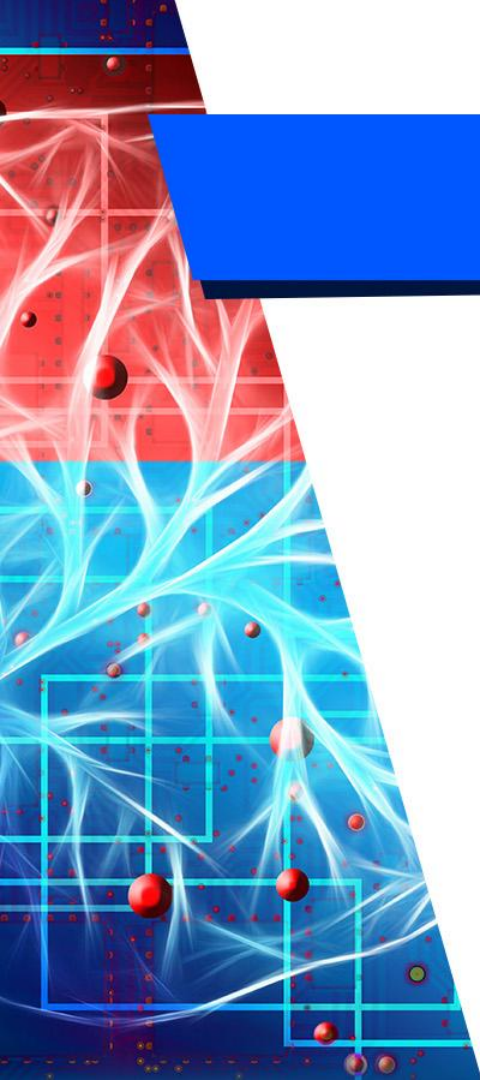




# Recurrencia RNN

- Las RNN son un tipo de red neuronal diseñada para procesar secuencias de datos. A diferencia de las redes neuronales feedforward tradicionales, las RNN tienen bucles que permiten que la información persista. En una RNN, la salida de una capa se almacena y se vuelve a usar como entrada para la siguiente capa. Esto permite que las RNN mantengan un "estado" que puede capturar información de entradas anteriores en la secuencia.
- **Problemas de las RNN:**
  - **Desvanecimiento del gradiente:** A medida que se entrena la red, los gradientes pueden volverse extremadamente pequeños, lo que dificulta el aprendizaje de dependencias a largo plazo.
  - **Explosión del gradiente:** Los gradientes pueden volverse extremadamente grandes, lo que lleva a actualizaciones inestables del modelo.



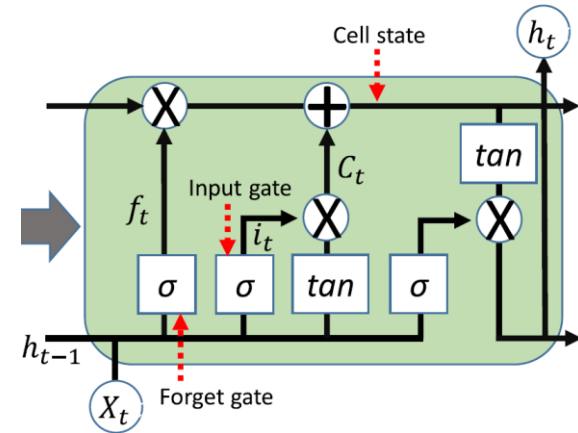
# Recurrencia LSTM

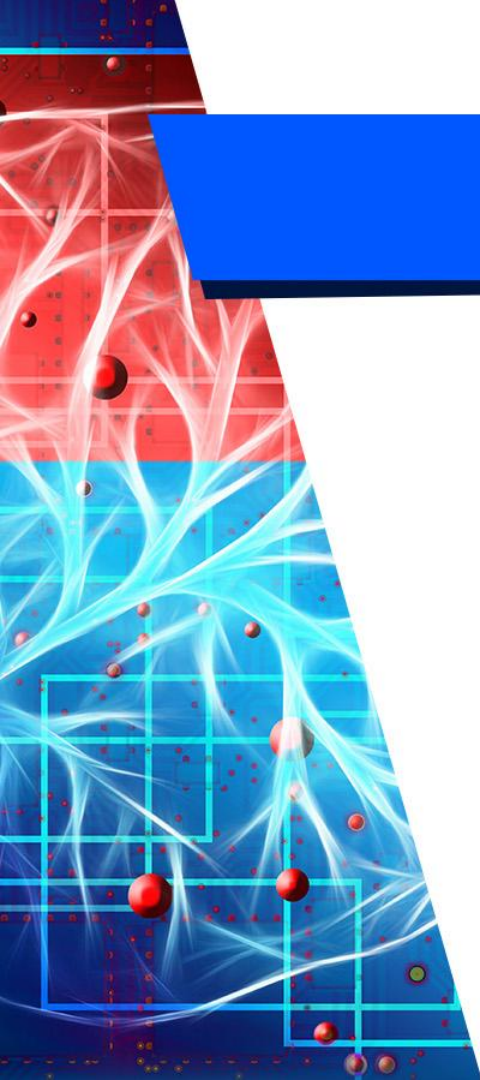
- Las LSTM son una variante de las RNN diseñadas para superar los problemas de desvanecimiento y explosión del gradiente. Introducen un mecanismo de "celdas de memoria" y "puertas" que regulan el flujo de información.
- **Componentes clave de las LSTM:**
  - **Celda de memoria:** Almacena información durante largos períodos de tiempo.
  - **Puerta de entrada:** Controla la cantidad de información nueva que se almacena en la celda.
  - **Puerta de olvido:** Controla la cantidad de información antigua que se olvida de la celda.
  - **Puerta de salida:** Controla la cantidad de información de la celda que se usa como salida.
- **Ventajas de las LSTM:**
  - Pueden capturar dependencias a largo plazo en los datos secuenciales.
  - Son más efectivas que las RNN estándar para secuencias largas y complejas.

# Celda LSTM

Los estados ocultos permiten a la LSTM recordar información de la secuencia de entrada en el corto plazo. Esto es fundamental para tareas secuenciales como traducción, análisis de texto o reconocimiento de voz.

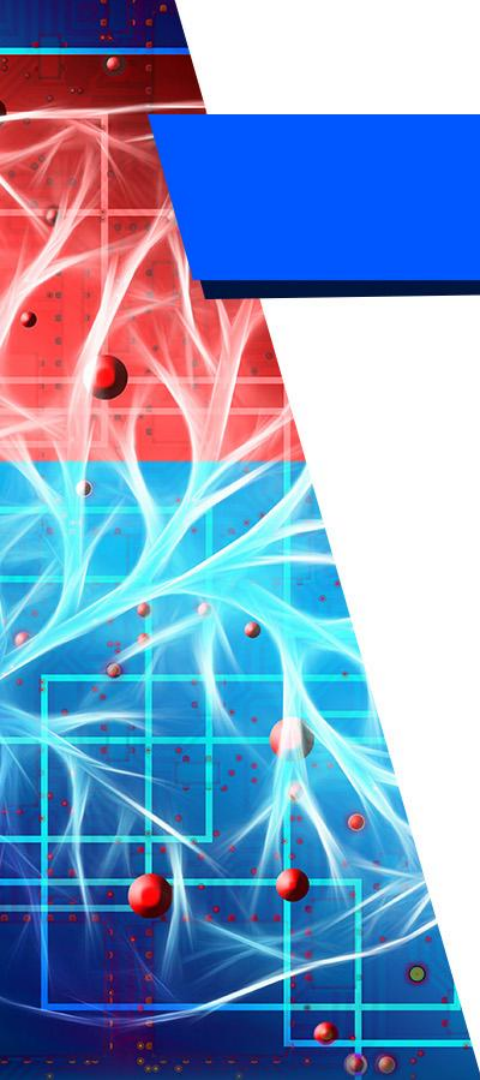
En cada paso de tiempo, la LSTM toma como entrada el vector de la entrada actual y el estado oculto del paso anterior. Esto le permite decidir qué información conservar, actualizar o olvidar.





# Recurrencia GRU

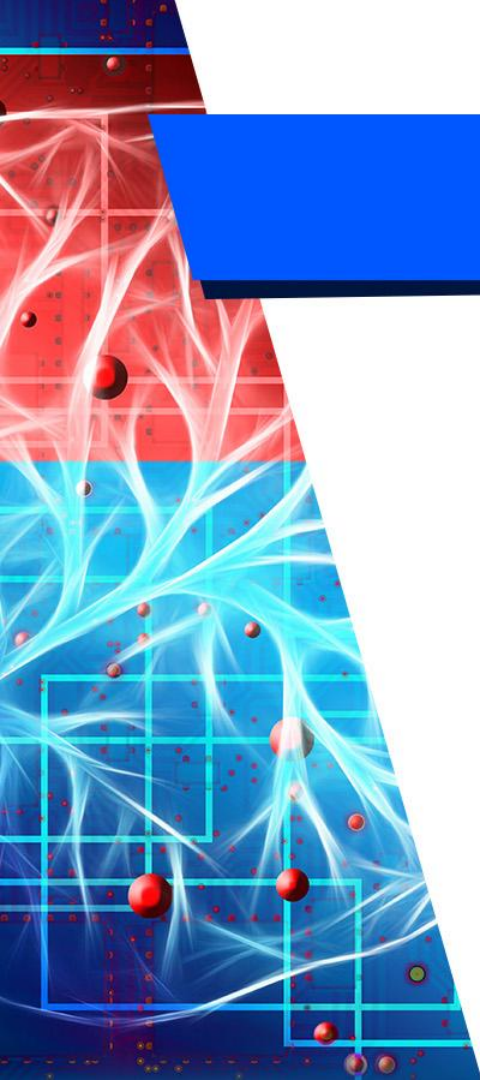
- Las GRU son otra variante de las RNN que, al igual que las LSTM, introducen mecanismos para controlar el flujo de información. Sin embargo, son más simples que las LSTM, ya que combinan algunas de las puertas.
- **Componentes clave de las GRU:**
  - **Puerta de actualización:** Controla la cantidad de información nueva que se almacena en la celda, similar a la combinación de la puerta de entrada y la puerta de olvido en las LSTM.
  - **Puerta de reinicio:** Controla la cantidad de información antigua que se olvida.
- **Ventajas de las GRU:**
  - Tienen menos parámetros que las LSTM, lo que las hace más rápidas de entrenar y menos propensas al sobreajuste.
  - Capturan dependencias a largo plazo de manera efectiva, similar a las LSTM.



# Comparativa

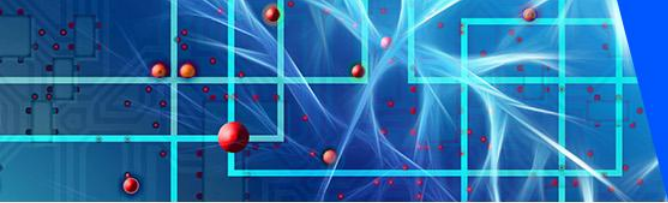
| Característica                 | RNN                                         | LSTM                                                      | GRU                                                                |
|--------------------------------|---------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------|
| Mecanismo de Memoria           | Estado oculto                               | Celdas de memoria con puertas de entrada, olvido y salida | Estado oculto con puertas de actualización y reinicio              |
| Problemas                      | Desvanecimiento/<br>explosión del gradiente | Mitigados por el diseño de las celdas de memoria          | Mitigados por el diseño de las puertas de actualización y reinicio |
| Complejidad                    | Baja                                        | Alta                                                      | Media                                                              |
| Velocidad de Entrenamiento     | Rápida                                      | Lenta                                                     | Más rápida que LSTM                                                |
| Número de Parámetros           | Menos                                       | Más                                                       | Menos que LSTM                                                     |
| Captura de Dependencias Largas | No efectiva                                 | Muy efectiva                                              | Efectiva                                                           |





# Consideraciones

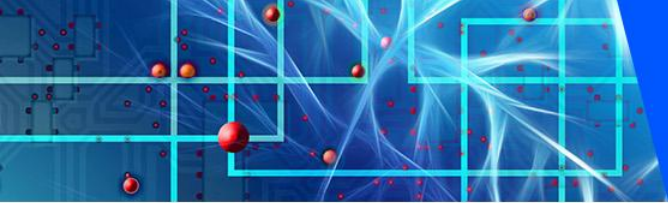
- Este tipo de modelos trabajan con ventanas temporales de datos de distintas variables. Podemos tener varias variables, pero para cada una de ellas necesitamos conocer su serie temporal.
- Si en alguna de las variables no tenemos suficiente información, tendremos que establecer algún valor. Podemos aplicar algún filtrado o bien dejar los valores a 0 si son totalmente desconocidos.



# ¿Cuándo utilizarlas?

Elegir entre LSTM, GRU y RNN depende de varios factores relacionados con tu problema específico y los datos con los que trabajas. Aquí hay una guía sobre cuándo usar cada uno:

- **RNN (Redes Neuronales Recurrentes)**
  - Uso: Cuando tienes secuencias cortas y no necesitas capturar relaciones a largo plazo.
  - Ventajas: Más simple y rápido de entrenar.
  - Desventajas: Problemas de desvanecimiento y explosión del gradiente, lo que dificulta el aprendizaje de dependencias a largo plazo.
- **LSTM (Long Short-Term Memory)**
  - Uso: Cuando necesitas manejar dependencias a largo plazo en tus datos, como en la traducción automática, el análisis de texto o la generación de secuencias.
  - Ventajas: Mejora la capacidad de recordar información durante períodos más largos gracias a sus puertas de entrada, olvido y salida.
  - Desventajas: Más complejo y puede ser más lento de entrenar que una RNN simple.
- **GRU (Gated Recurrent Unit)**
  - Uso: Similar a LSTM, pero a menudo se prefiere en situaciones donde se busca un balance entre complejidad y rendimiento.
  - Ventajas: Menos parámetros que LSTM, lo que puede resultar en un entrenamiento más rápido y un mejor rendimiento en algunos casos. Maneja bien las dependencias a largo plazo.
  - Desventajas: A veces, puede no capturar ciertas dinámicas que LSTM puede manejar mejor.
- **Resumen:**
  - Usa RNN para tareas simples o cuando trabajas con secuencias cortas.
  - Usa LSTM cuando tu tarea requiere aprender dependencias a largo plazo.
  - Usa GRU si buscas un modelo más ligero que aún capture bien las dependencias a largo plazo.



# Ejercicio práctico (I)

- Predicción de la temperatura de una ciudad a partir de la serie temporal de datos de temperatura previa, humedad, velocidad del viento y presión atmosférica.
- Dataset:  
<https://www.kaggle.com/datasets/sumanthvrao/daily-climate-time-series-data>



# Ejercicio práctico (II)

- Carga y visualización de datos (I):

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM
from sklearn.preprocessing import MinMaxScaler

# Especifica la ruta a tu fichero CSV
datos_train = 'DailyDelhiClimateTrain.csv'
datos_test = 'DailyDelhiClimateTest.csv'

# Cargar el fichero CSV
df_train = pd.read_csv(datos_train)
df_test = pd.read_csv(datos_test)

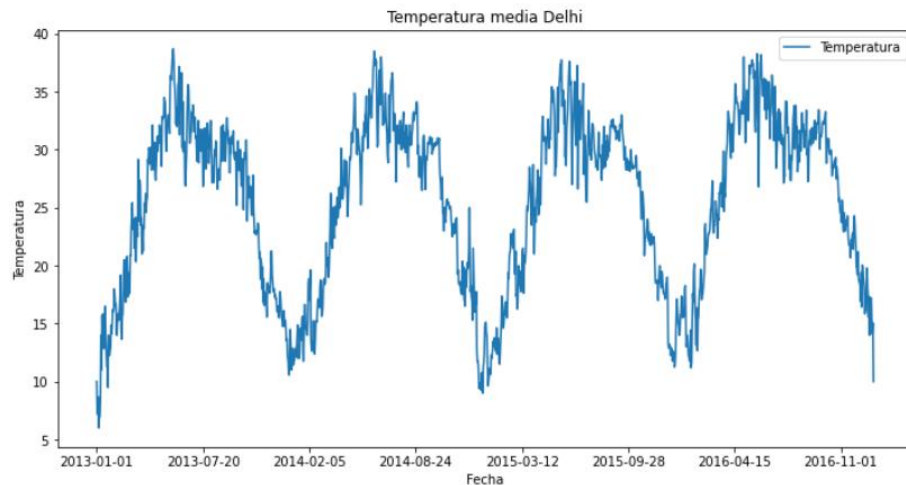
# Mostrar las primeras filas del DataFrame para verificar que se ha cargado correctamente
print(df_train.head())

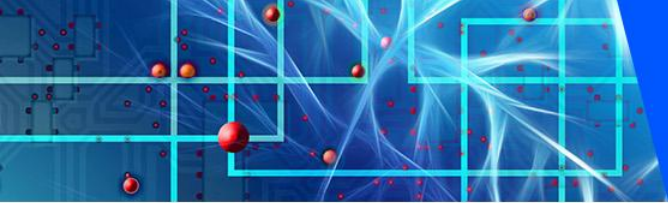
# Convertir la fecha en una característica numérica para el procesamiento
df_train['date_fdate'] = pd.to_datetime(df_train['date']) # crear columna datetime
df_train['date_numeric'] = df_train['date_fdate'].apply(lambda x: x.timestamp()) # crear numérica
```

# Ejercicio práctico (III)

- Carga y visualización de datos (II):

```
# Plot de magnitudes
plt.figure(figsize=(12, 6))
plt.plot(df_train['date'], df_train['meantemp'], label='Temperatura')
# Ajustar los ticks del eje x para mostrar solo 10 elementos
plt.gca().xaxis.set_major_locator(plt.MaxNLocator(10))
plt.xlabel('Fecha')
plt.ylabel('Temperatura')
plt.title('Temperatura media Delhi')
plt.legend()
plt.show()
```





# Ejercicio práctico (IV)

- Preprocesamiento de datos: Utilizamos MinMaxScaler para ajustar las columnas numéricas en intervalo  $[0,1]$

```
#####  
# Preparación de los datos  
#####  
scaler1=MinMaxScaler()  
scaler2=MinMaxScaler()  
scaler3=MinMaxScaler()  
scaler4=MinMaxScaler()
```

# Ejercicio práctico (V)

```
def prepararDataset(df):  
    global scaler1, scaler2, scaler3, scaler4  
    # Ponemos los valores en rango [0,1]  
    meantemp = scaler1.fit_transform(df[['meantemp']])  
    humidity = scaler2.fit_transform(df[['humidity']])  
    windspeed = scaler3.fit_transform(df[['wind_speed']])  
    meanpressure = scaler4.fit_transform(df[['meanpressure']])  
  
    # Juntamos las columnas  
    data = np.concatenate((meantemp, humidity, windspeed, meanpressure), axis=1)  
  
    # Creamos ventanas de 30 elementos.  
    # Aquí seleccionamos grupos de 31 elementos, donde la última fila sera la salida predicha  
    x=[]  
    for i in range(data.shape[0]-30):  
        row = data[i:i+31]  
        x.append(row)  
  
    x=np.array(x)  
    X_out = x[:, :-1] # Primeras 30 mediciones de cada ventana son la entrada  
    Y_out = x[:, -1, 0] # La medición 31 es la salida, y seleccionamos sólo la temperatura.  
  
    X_out = X_out.reshape(-1,4,30) # creamos ventanas de 30 elementos para cada uno de los 4 valores  
  
    return X_out, Y_out
```

- Preprocesamiento de datos:  
Creamos ventanas de 30 valores para cada variable.

# Ejercicio práctico (VI)

- Definimos nuestro modelo y lo entrenamos. Tenemos que especificar el número de variables y el tamaño de la ventana

```
X_train, Y_train = prepararDataset(df_train)
X_test, Y_test = prepararDataset(df_test)

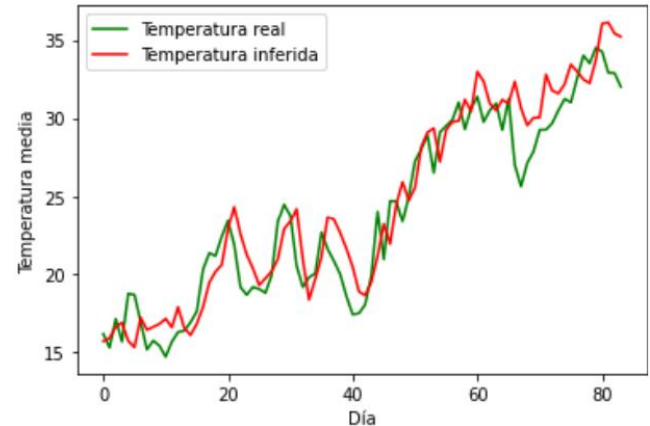
#####
# Definir modelo y entrenarlo
#####
model=Sequential()
model.add(LSTM(100, input_shape=(4, 30), return_sequences=False, recurrent_dropout=0.2))
#model.add(LSTM(100, input_shape=(4, 30), return_sequences=True))
#model.add(LSTM(100, return_sequences=True, recurrent_dropout=0.2))
#model.add(LSTM(100, return_sequences=True, recurrent_dropout=0.2))
#model.add(LSTM(100, return_sequences=False, recurrent_dropout=0.2))
model.add(Dense(1, activation='linear'))
model.compile(optimizer='adam', loss='mean_squared_error')
model.summary()
model.fit(X_train, Y_train, batch_size=32, epochs=100)
```

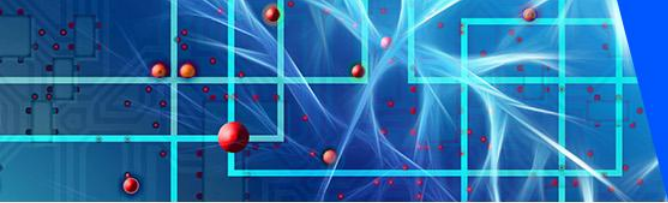


# Ejercicio práctico (VII)

- Realizamos la predicción del conjunto de test y visualizamos el resultado gráficamente. Cada 30 días, predecimos la temperatura

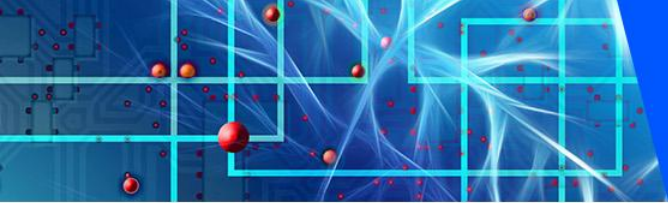
```
#####  
# Predicción de test y visualización de resultado  
#####  
predictions = model.predict(X_test)  
predictions = scaler1.inverse_transform(predictions) # convertir a escala original  
  
# convertir Y_test a la misma dimensión que las predicciones  
Y_test = Y_test.reshape(-1,1)  
Y_test = scaler1.inverse_transform(Y_test)  
  
plt.plot(Y_test, label='Temperatura real', color='green')  
plt.plot(predictions, label='Temperatura inferida', color='red')  
plt.legend(loc='best')  
plt.xlabel('Día')  
plt.ylabel('Temperatura media')  
plt.show()
```





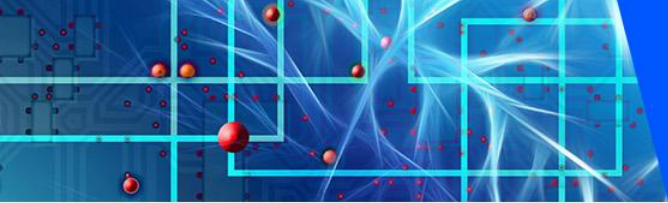
# Más allá de la recurrencia (I)

- Además de las capas LSTM o GRU normales, es posible utilizar capas LSTM o GRU bidireccionales.
- Una LSTM /GRU normal procesa las secuencias de datos de manera unidireccional, es decir, lee los datos de entrada en una única dirección, generalmente de izquierda a derecha (pasado hacia el futuro).
- Una LSTM/GRU bidireccional procesa las secuencias en ambas direcciones, es decir, lee los datos de entrada tanto de izquierda a derecha (pasado al futuro) como de derecha a izquierda (futuro al pasado).
- Esto permite que el modelo capture información tanto del contexto previo como del contexto futuro, lo que resulta útil cuando toda la secuencia está disponible, especialmente en tareas donde la comprensión completa de la secuencia es importante (como en traducción automática, etiquetado de secuencias, o reconocimiento de voz).



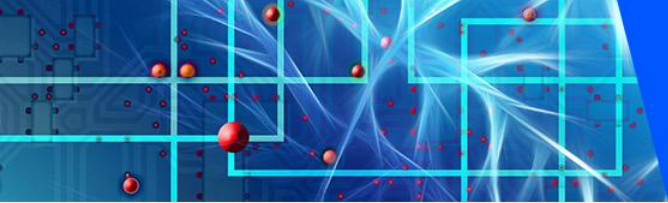
# NLP (Procesamiento del Lenguaje Natural) (I)

- El Procesamiento del Lenguaje Natural (NLP) es una rama de la inteligencia artificial que permite a las computadoras entender, interpretar y generar lenguaje humano, facilitando tareas como la traducción automática, análisis de sentimientos, chatbots y resumen de texto.
- La recurrencia se puede utilizar en problemas de procesamiento del lenguaje natural. Un ejemplo es la traducción de textos.
- El modelo Seq2Seq (Sequence-to-Sequence) es una arquitectura de redes neuronales diseñada para transformar una secuencia de entrada en una secuencia de salida de longitud variable. Este tipo de modelo es especialmente útil en tareas donde la entrada y la salida son secuencias, como la traducción automática, el resumen de textos o la generación de respuestas en chatbots.
- Existen modelos como GPT que utilizan una arquitectura utilizando Transformers y mecanismos de atención.



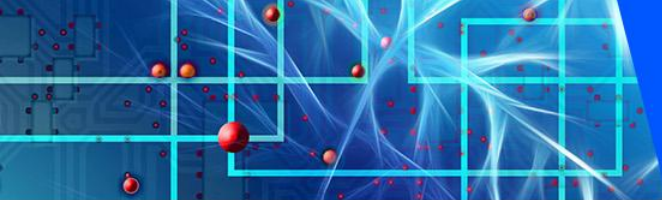
# NLP (Procesamiento del Lenguaje Natural) (II)

- Componentes principales del modelo Seq2Seq
  - Codificador (Encoder):
    - Recibe la secuencia de entrada y la procesa para generar una representación codificada (un vector o conjunto de vectores) que captura la información de toda la secuencia.
    - El codificador suele ser una red neuronal recurrente (RNN), como una LSTM (Long Short-Term Memory) o una GRU (Gated Recurrent Unit). Su propósito es generar un "estado oculto" que resume el contexto de la secuencia de entrada.
  - Decodificador (Decoder):
    - Toma la representación generada por el codificador (estado oculto final) como punto de partida y genera la secuencia de salida de manera recurrente.
    - El decodificador también suele ser una red neuronal recurrente (LSTM, GRU, etc.) que, en cada paso, predice la siguiente palabra de la secuencia de salida basándose en el estado oculto recibido y en las palabras que ha generado hasta ese punto.
  - Estado oculto:
    - Es la conexión entre el codificador y el decodificador. Al final de la secuencia de entrada, el estado oculto del codificador se pasa al decodificador como punto de inicio. Este estado contiene la información que el decodificador necesita para generar la secuencia de salida.



# NLP (Procesamiento del Lenguaje Natural) (III)

- Flujo de trabajo del Seq2Seq:
  - Codificación:
    - El codificador procesa la secuencia de entrada palabra por palabra (o símbolo por símbolo, dependiendo del caso). En cada paso de tiempo, actualiza su estado oculto.
    - Después de procesar toda la secuencia de entrada, el estado oculto del codificador contiene un resumen de toda la secuencia.
  - Decodificación:
    - El decodificador comienza con el estado oculto final del codificador como su entrada inicial y genera secuencias paso a paso. En cada paso, predice una palabra/símbolo de la secuencia de salida.
    - Durante el entrenamiento, el decodificador también puede utilizar la secuencia de salida esperada (método llamado teacher forcing), lo que significa que en cada paso de tiempo recibe la palabra correcta de la secuencia de salida como entrada, junto con el estado oculto.
    - Durante la inferencia (generación de secuencias), el decodificador solo utiliza las palabras que él mismo ha generado como entrada para el siguiente paso.



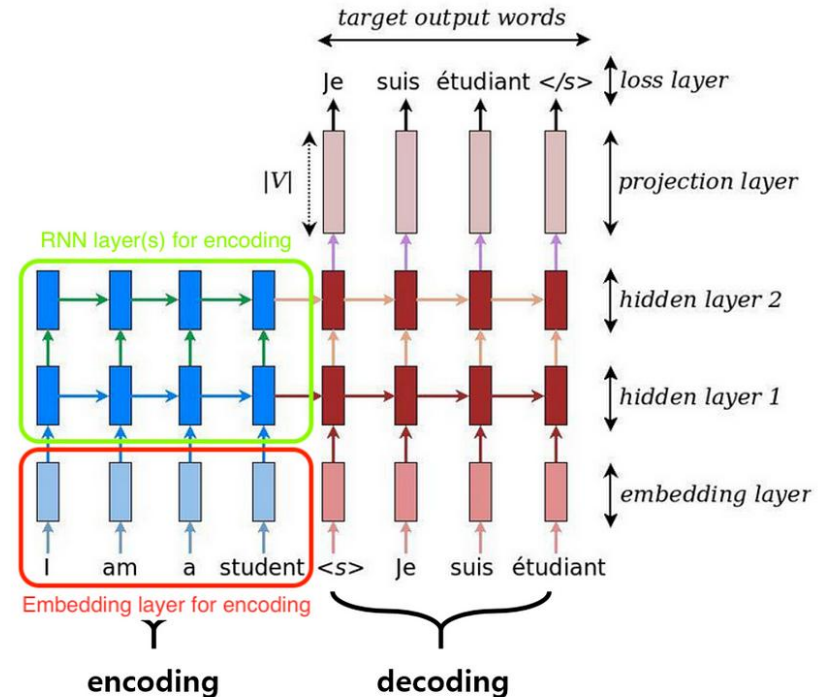
# NLP (Procesamiento del Lenguaje Natural) (IV)

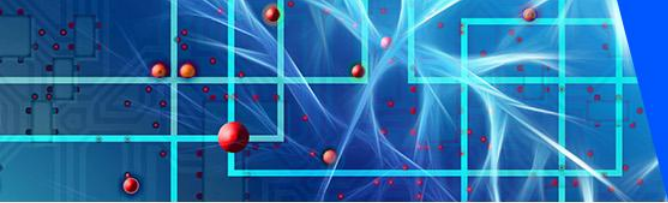
- Flujo de trabajo del Seq2Seq (inferencia):
  - **El modelo seq2seq genera una única palabra en cada predicción durante la inferencia.**
  - Por ese motivo tenemos que llamar varias veces a la predicción para generar palabras hasta que obtengamos la palabra de finalización, por ejemplo, FIN o </s>. A mayores, se suele especificar un tamaño máximo de frase de salida.



# NLP (Procesamiento del Lenguaje Natural) (V)

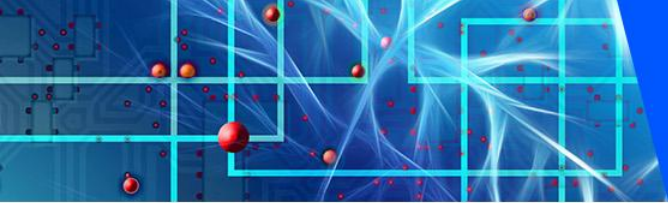
- Flujo de trabajo del Seq2Seq:
  - Ejemplo de entrada durante el entrenamiento en método de teacher forcing:
    - encoder\_input\_data: ["how", "are", "you"] (no desplazada)
    - decoder\_input\_data: ["START", "hola"] (no desplazada)
    - decoder\_target\_data: ["hola", "FIN"] (desplazada a la izquierda)





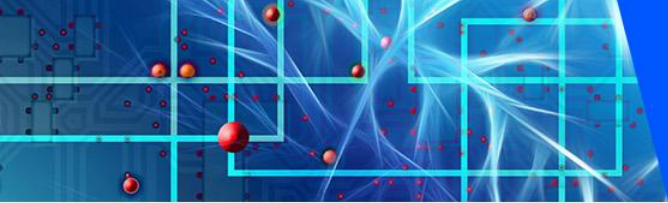
# NLP (Procesamiento del Lenguaje Natural) (VII)

- La tokenización es el proceso de dividir texto en unidades más pequeñas llamadas "tokens", que pueden ser palabras, subpalabras o caracteres. Es esencial en el procesamiento del lenguaje natural, ya que permite a los modelos entender y manipular texto de manera estructurada. En los modelos de aprendizaje automático, dichos tokens suelen ser números para poder entrar en el modelo.
- El embedding asigna un vector único a cada token, representando su significado. Cada token se convierte en su vector correspondiente de manera independiente, sin conexiones directas entre los tokens previos. Sin embargo, en capas posteriores, como LSTM o capas densas, esos vectores pueden interactuar y aprender patrones en función de las secuencias.



# NLP (Procesamiento del Lenguaje Natural) (VIII)

- El padding es una técnica utilizada en redes neuronales para hacer que todas las secuencias de datos tengan la misma longitud. Se añade un valor (generalmente ceros) al principio o al final de las secuencias más cortas para que coincidan con la longitud de la secuencia más larga en un conjunto de datos. Esto es necesario para procesar secuencias en lotes de tamaño uniforme.



# NLP (Procesamiento del Lenguaje Natural) (IX)

- Tokenización convierte texto en unidades discretas.
- Embedding convierte esos tokens en vectores densos.
- Capa Densa procesa esos vectores para aprender y hacer predicciones.
- En el modelo seq2seq se agregan palabras de control al inicio y final de la secuencia que se está generando. Por ejemplo, con los Tokens "START" y "FIN". Si estamos haciendo un traductor de inglés a español, se agregan al inicio y al final de cada oración en español.
- En la función de decodificación, una vez entrenado el modelo, se utiliza el token "START" para iniciar la generación y se busca el token "FIN" para detenerse. Estos tokens indica al decodificador que comience la generación de la frase. Sin un token inicial, el modelo no sabría cuándo empezar a producir la salida. El modelo genera palabras hasta que encuentra el token "FIN" o alcanza una longitud máxima.

# NLP (Procesamiento del Lenguaje Natural) (X)

## • Traducción Inglés → Español

```
import numpy as np
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, Dense, Embedding
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import pandas as pd
```

```
# Cargar el archivo CSV
df = pd.read_csv('spanish-to-english.csv')
```

```
PALABRA_INICIO = "xxxxxx"
PALABRA_FINAL = "yyyyyy"
```

```
# Extraer las columnas específicas
spanish_sentences = df.iloc[:, 1].tolist() # Tercera columna (índice 2)
english_sentences = df.iloc[:, 2].tolist() # Segunda columna (índice 1)
```

```
# Reducimos dataset para experimentar
spanish_sentences = spanish_sentences[:20000]
english_sentences = english_sentences[:20000]
```

```
# Agregar tokens "START" y "FIN" a las oraciones en español
spanish_sentences = [PALABRA_INICIO + ' ' + s + ' ' + PALABRA_FINAL for s in spanish_sentences]
```

Carga de Dataset:

<https://www.kaggle.com/datasets/hunhnguynphe/spanish-to-english-dataset>

Tokenización



# Tokenización y preprocesamiento de datos

```
# Tokenizador para el inglés
english_tokenizer = Tokenizer()
english_tokenizer.fit_on_texts(english_sentences)
english_sequences = english_tokenizer.texts_to_sequences(english_sentences)
english_vocab_size = len(english_tokenizer.word_index) + 1
max_encoder_seq_length = max(len(seq) for seq in english_sequences)
```

# Tokenizador para el español

```
spanish_tokenizer = Tokenizer()
spanish_tokenizer.fit_on_texts(spanish_sentences)
spanish_sequences = spanish_tokenizer.texts_to_sequences(spanish_sentences)
spanish_vocab_size = len(spanish_tokenizer.word_index) + 1
max_decoder_seq_length = max(len(seq) for seq in spanish_sequences)
```

# Padding para las secuencias

```
encoder_input_data = pad_sequences(english_sequences, maxlen=max_encoder_seq_length,
padding='post')
decoder_input_data = pad_sequences(spanish_sequences, maxlen=max_decoder_seq_length,
padding='post')
```

# Creamos las secuencias de salida (desplazadas)

```
decoder_target_data = np.zeros_like(decoder_input_data)
decoder_target_data[:, :-1] = decoder_input_data[:, 1:]
```

# Esto nos sirve para eliminar la palabra final de la entrada del decoder

```
for i, x in enumerate(decoder_input_data):
    x = list(x)
    try:
        decoder_input_data[i][x.index(0) - 1] = 0
    except:
        continue
```

# NLP (Procesamiento del Lenguaje Natural) (XI)

- Traducción Inglés → Español

Definición de modelo  
y entrenamiento



```
# Definición del modelo Seq2Seq
```

```
# Codificador (Encoder)
```

```
encoder_inputs = Input(shape=(None,))
```

```
encoder_embedding = Embedding(input_dim=english_vocab_size, output_dim=64)(encoder_inputs)
```

```
encoder_lstm = LSTM(64, return_state=True)
```

```
encoder_outputs, state_h, state_c = encoder_lstm(encoder_embedding)
```

```
encoder_states = [state_h, state_c]
```

```
# Decodificador (Decoder)
```

```
decoder_inputs = Input(shape=(None,))
```

```
decoder_embedding = Embedding(input_dim=spanish_vocab_size, output_dim=64)(decoder_inputs)
```

```
decoder_lstm = LSTM(64, return_sequences=True, return_state=True)
```

```
decoder_outputs, _, _ = decoder_lstm(decoder_embedding, initial_state=encoder_states)
```

```
decoder_dense = Dense(spanish_vocab_size, activation="softmax")
```

```
decoder_outputs = decoder_dense(decoder_outputs)
```

```
# Modelo Seq2Seq completo
```

```
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
```

```
# Compilación y entrenamiento del modelo
```

```
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
```

```
model.summary()
```

```
# Entrenamos el modelo. Ajustar épocas
```

```
model.fit([encoder_input_data, decoder_input_data], decoder_target_data, batch_size=64, epochs=50)
```



# NLP (Procesamiento del Lenguaje Natural) (XII)

- Traducción Inglés → Español

Inferencia y función de  
decodificación



# Inferencia

```
encoder_model = Model(encoder_inputs, encoder_states)
```

# Modelo del decoder

```
decoder_state_input_h = Input(shape=(64,))
```

```
decoder_state_input_c = Input(shape=(64,))
```

```
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]
```

```
decoder_outputs, state_h, state_c = decoder_lstm(  
    decoder_embedding, initial_state=decoder_states_inputs)
```

```
decoder_states = [state_h, state_c]
```

```
decoder_outputs = decoder_dense(decoder_outputs)
```

```
decoder_model = Model([decoder_inputs] + decoder_states_inputs, [decoder_outputs] +  
    decoder_states)
```

# Esta función genera de palabra en palabra

```
def decode_sequence(input_seq):
```

```
    states_value = encoder_model.predict(input_seq)
```

# Inicializar la secuencia del decodificador con el token de inicio

```
target_seq = np.zeros((1, 1))
```

```
target_seq[0, 0] = spanish_tokenizer.word_index[PALABRA_INICIO]
```

```
decoded_sentence = []
```

while True:

# Predecir el siguiente token

```
output_tokens, h, c = decoder_model.predict([target_seq] + states_value)
```

```
sampled_token_index = np.argmax(output_tokens[0, -1, :])
```

# Convertir el índice de vuelta a la palabra

```
sampled_word = spanish_tokenizer.index_word.get(sampled_token_index, "")
```

# Condición de finalización: buscar el token final o longitud máxima

```
if sampled_word == PALABRA_FINAL or len(decoded_sentence) > max_decoder_seq_length:  
    break
```

```
elif sampled_word in decoded_sentence: # Evitar repeticiones de palabras  
    break
```

if sampled\_word:

```
    decoded_sentence.append(sampled_word)
```

# Actualizar la secuencia de entrada para el siguiente paso

```
target_seq = np.zeros((1, 1))
```

```
target_seq[0, 0] = sampled_token_index
```

# Actualizar los estados ocultos

```
states_value = [h, c]
```

```
return ' '.join(decoded_sentence)
```

# NLP (Procesamiento del Lenguaje Natural) (XIII)

- Traducción Inglés→Español

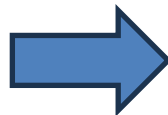
Test



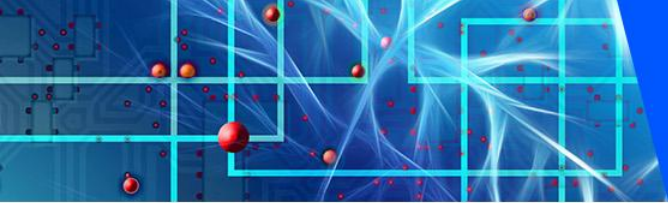
# Probamos con una nueva secuencia de entrada

```
test_sentence = ["how are you"]  
test_sequence = english_tokenizer.texts_to_sequences(test_sentence)  
test_sequence_padded = pad_sequences(test_sequence,  
maxlen=max_encoder_seq_length, padding='post')
```

```
decoded_sentence = decode_sequence(test_sequence_padded)  
print(f'Frase en inglés: {test_sentence[0]}')  
print(f'Traducción al español: {decoded_sentence}')
```

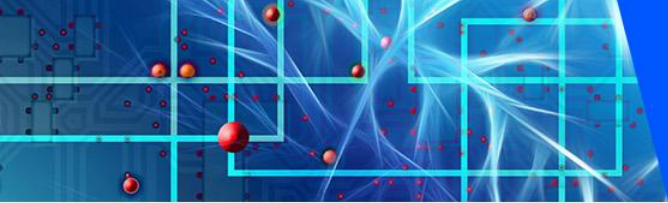


```
Epoch 50/50  
313/313 _____ 9s 30ms/step - loss: 0.1966  
1/1 _____ 0s 87ms/step  
1/1 _____ 0s 83ms/step  
1/1 _____ 0s 13ms/step  
1/1 _____ 0s 14ms/step  
Frase en inglés: how are you  
Traducción al español: ¿cómo son
```



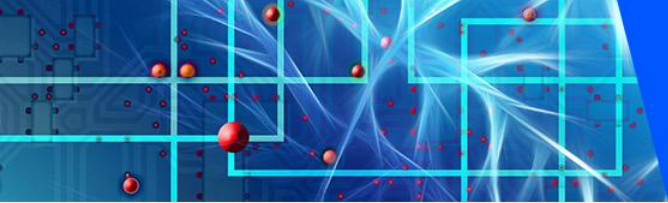
# NLP (Procesamiento del Lenguaje Natural) (XIV)

- Uno de los problemas al utilizar LSTM es que en tareas donde la secuencia de entrada es larga, el vector codificado puede perder información importante. Para abordar esto, se utiliza una técnica llamada Atención (Attention) en otros modelos como GPT.
- GPT, que significa "Generative Pre-trained Transformer," es un modelo desarrollado por OpenAI. Se basa en la arquitectura de Transformer, presentada por primera vez en el artículo "Attention is All You Need" en 2017. Desde su introducción, OpenAI ha lanzado varias versiones de GPT, cada una mejorando en capacidad y rendimiento.
- La idea principal detrás de GPT es que se pre-entrena en grandes cantidades de texto de diversas fuentes antes de ajustarse para tareas específicas. Este enfoque permite que el modelo entienda y genere texto de manera coherente y relevante.



# Ejercicios

- La recurrencia se puede utilizar en problemas de predicción de valores en series temporales, de detección de anomalías en conjuntos de datos temporales e incluso de procesamiento de vídeo, considerando cada frame un elemento de la serie temporal.
- **EJERCICIO PRÁCTICO:**
  - Busca un conjunto de datos temporal, por ejemplo: la evolución del precio de la vivienda, datos de sensores de una máquina con anomalías, datos sísmicos de una zona para predicción de terremotos, etc. Puedes encontrar en Kaggle.
  - Desarrolla un modelo LSTM y compáralo con un modelo GRU o RNN. Revisa si funciona mejor con más o menos capas de recurrencia.
  - Visualiza o evalúa los resultados con un conjunto de test.



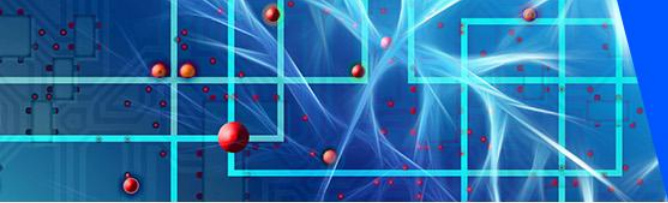
# Ejercicio 1

- Descarga el valor de cierre de alguna acción financiera entre dos fechas y realiza una predicción de su comportamiento en los próximos días. Evalúa su resultado en un intervalo de tiempo.

```
import yfinance as yf
```

```
# Descargar datos históricos de una acción (AAPL, GOOGL, etc.)  
data = yf.download('AAPL', start='2020-01-01', end='2022-01-01')
```

```
# Usar solo la columna de precios de cierre  
data = data[['Close']]
```



# Ejercicio 2

- Desarrolla un programa de detección de anomalías en alguno de los siguientes dos datasets:
  - NASA's Prognostics Data Repository  
<https://www.nasa.gov/content/prognostics-center-of-excellence-data-set-repository>
    - Descripción: Incluye datos de sensores para varios sistemas, con fallos y anomalías etiquetadas. Utilizado para la detección de fallos y mantenimiento predictivo.
    - Características: Datos de sensores con fallos y condiciones anómalas para sistemas de maquinaria y aeronaves.
    - Elegir uno de los conjuntos de datos
  - ECG5000 Dataset  
<https://www.timeseriesclassification.com/description.php?Dataset=ECG5000>
    - Descripción: Contiene datos de electrocardiogramas (ECG) con señales de diferentes tipos de anomalías cardíacas. Ideal para la detección de anomalías en señales biomédicas.
    - Características: Datos de ECG con etiquetas de diferentes tipos de anomalías cardíacas.